AgentGate

Day 1 Notes: Policy Service and Core Spring Concepts

Monday, 26 October 2026 | Week 1 of 10

Contents

1 What we built today **2**

2 Project setup **2**

2.1 Why a parent POM . . . 2

2.2 Infrastructure (Docker Compose) . . . 2

3 Business logic of the Policy Service **3**

3.1 A policy is a rule . . . 3

3.2 The three operations . . . 3

3.3 The check rule . . . 3

3.4 Live test results . . . 4

4 The layers **4**

5 Important concepts **4**

5.1 Entity vs repository . . . 4

5.2 How column names are mapped . . . 5

5.3 Derived queries . . . 5

5.4 DTO (Data Transfer Object) . . . 5

5.5 Records . . . 5

5.6 Service layer, interface and implementation . . . 6

5.7 Dependency injection and the constructor . . . 6

5.8 `@Transactional` . . . 6

5.9 `extends` vs `implements`; inheritance vs polymorphism . . . 7

5.10 SQL types used . . . 7

5.11 Small but useful . . . 7

6 Mistakes and lessons **8**

7 Next steps **8**

8 Interview answers from today **8**

1 What we built today

AgentGate is a gateway that sits between AI agents and company tools. Every agent action must pass three checks: *who is the agent*, *is it allowed*, and *does the company still have budget*. Today we built the second check: the **Policy Service**.

Area	Result
Project setup (Week 0)	Maven multi-module project, parent POM, five service modules plus `common`; Docker Compose with Postgres 16, Redis 7 and Kafka (KRaft mode). All three containers healthy.
Database	Flyway migration `V1__create_policies.sql`; separate database per service.
Code	Entity, repository, four DTO records, service interface and implementation, REST controller.
Tested live	Create rule (201), list rules (200), check (200), validation error (400), and "DENY wins" proven with real HTTP calls.
Still to do (Week 1)	Global exception handler, Caffeine cache, Mockito unit tests.

2 Project setup

2.1 Why a parent POM

The parent `pom.xml` is the one file all six modules share. It holds settings once so they are not copied into every service.

- **Module list**: one `mvn verify` at the root builds everything in the right order (`common` first).
- **One Spring Boot version** (3.5.6) for every service, so library versions never differ.
- **One Java version** (21).
- `dependencyManagement`: modules list a library without a version; the version comes from the parent.
- `pluginManagement`: shared compiler setup (MapStruct).

A module uses the parent (`<parent>`) to get *settings*, and uses `common` as a dependency (`<dependency>`) to get *code*. These are different things.

2.2 Infrastructure (Docker Compose)

- **Postgres**: one server, four databases (`auth_db`, `policy_db`, `budget_db`, `orchestrator_db`). No service reads another service's database.
- **Redis**: AOF persistence on with `appendfsync everysec`, so a restart loses at most about one second of writes.
- **Kafka**: one node acting as broker and controller (KRaft, no ZooKeeper). Apps on the laptop use `localhost:9092`; apps inside Compose use `kafka:29092`.

3 Business logic of the Policy Service

The Policy Service answers one question: *"Is this agent allowed to use this tool?"* In the full flow the Orchestrator asks it before any budget is spent. A "no" stops the action with status `DENIED`.

3.1 A policy is a rule

One row in the `policies` table is one rule, for example: *in tenant `acme`, agent `support-bot`, tool `send_email` → ALLOW.*

Field	Meaning
`tenantId`	Which customer company owns the rule. Rules of different companies stay separate.
`agentId`	Which AI agent the rule applies to.
`tool`	Which tool the agent wants to use.
`effect`	`ALLOW` or `DENY`.
`conditions`	Extra limits as JSON, e.g. weekdays only. Stored now, enforced later (Strategy pattern).

3.2 The three operations

1. **Create** – `POST /policies` returns 201 with the saved rule. The client cannot choose `id` or `createdAt`.
2. **List** – `GET /policies` returns all rules (admin).
3. **Check** – `POST /policies/check` is the real business logic.

3.3 The check rule

The service loads all rules for the exact tenant, agent and tool, then decides:

1. If **any** rule is `DENY` → not allowed (*denied by policy*).
2. Else if **any** rule is `ALLOW` → allowed (*allowed by policy*).
3. Else → not allowed (*no matching rule*).

Why these two principles matter

- **DENY always wins.** If both an ALLOW and a DENY exist, access is blocked. A mistake in the rules should lock the door, not open it. The result does not depend on which rule was saved first. (Our first version returned on the first row it saw, which broke this. A good example of a security bug.)
- **Default deny.** If nothing matches, the answer is no. This is the *zero trust* idea: never allow because of silence.

3.4 Live test results

Test	Result
Check with no rules	200, allowed = false, "no matching rule"
Create an ALLOW rule	201, server filled `id` and `createdAt`
Check again	allowed = true
Add a DENY rule, check again	allowed = false, "denied by policy"
Blank `tenantId`	400 (validation works)
`GET /policies`	200 with both rules

4 The layers

Every service follows Controller → Service → Repository. A layer never skips the next one, and the entity never leaves the service layer.

Layer	Our class	Its one job
Controller	`PolicyController`	Receive JSON, validate (`@Valid`), call the service, return the result. No rules here.
Service	`PolicyServiceImpl`	Business rules, transactions, entity ↔ DTO conversion.
Repository	`PolicyRepository`	Load and save data. Spring writes the code.
Entity	`Policy`	One object = one table row.
DTO	four records	Carry data in and out of the API.

Listing 1: Request flow for POST /policies/check

```
JSON body -> CheckRequest (validated) -> PolicyController.checkPolicy
          -> PolicyServiceImpl.check   -> PolicyRepository.
    findByTenantIdAndAgentIdAndTool
          -> database -> List<Policy>  -> CheckResponse -> JSON
```

5 Important concepts

5.1 Entity vs repository

An **entity** is the data: one Java object is one table row, with fields only. A **repository** is the tool that saves and loads entities. Think of the entity as a form and the repository as the filing cabinet.

	Entity	Repository
Is a	class	interface
Contains	fields (columns)	query methods
You write	fields and annotations	almost nothing; Spring generates it

```
@Entity @Table(name = "policies")
public class Policy {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    @Column(nullable = false) private String tenantId;
    ...
}

public interface PolicyRepository extends JpaRepository<Policy, Long> {
    List<Policy> findByTenantIdAndAgentIdAndTool(String tenantId, String agentId,
    String tool);
}
```

5.2 How column names are mapped

Hibernate reads each field of the entity. It uses `@Column(name=...)` if present; otherwise Spring Boot converts camelCase to snake_case (`tenantId` → `tenant_id`). With `ddl-auto: validate`, Hibernate checks that every column exists in the real table and refuses to start on a mismatch. Our typo `tennatId` would have failed exactly this way. The SQL file is the truth; the entity is a description that must agree with it. `@Column` also carries `nullable`, `length` and similar settings.

5.3 Derived queries

Spring reads the *method name* and builds the query: `find` (SELECT) `By` (conditions start) `TenantId And AgentId And Tool`. Names refer to **Java field names**, parameters match by **position**, and mistakes fail at startup. Resulting SQL:

```
SELECT id, tenant_id, agent_id, tool, effect, conditions, created_at
FROM policies WHERE tenant_id = ? AND agent_id = ? AND tool = ?
```

For custom queries use `@Query` with JPQL (entity and field names) or `nativeQuery = true` for Postgres features such as JSONB. `@Modifying` is required for UPDATE and DELETE.

5.4 DTO (Data Transfer Object)

A DTO is a plain object that carries data in and out of the API. The entity is the recipe card in the kitchen; the DTO is the plate served to the customer.

Why not return entities directly

- Clients could send or overwrite fields like `id` and `createdAt`.
- The API would be tied to the database design.
- Create and response need different fields.
- Validation annotations belong on the input object.
- JPA internals (lazy loading) can break JSON output.

Our four DTOs: `CreatePolicyRequest` (`@NotBlank` strings, `@NotNull Effect`), `PolicyResponse`, `CheckRequest`, `CheckResponse`. We corrected one: `CreatePolicyRequest` had `effect` as a `String`, a `@NotBlank` on a `Map`, and a client-supplied `createdAt`.

5.5 Records

A `record` is a short class for immutable data. Java generates the private final fields, constructor, accessors, `equals`, `hashCode` and `toString`. Accessors have *no* `get` prefix: `request.tenantId()`, not

`getTenantId()`. Entities cannot be records because JPA needs to change fields and needs an empty constructor.

```
public record CheckResponse(boolean allowed, String reason) { }
```

5.6 Service layer, interface and implementation

The service holds business logic. The interface (`PolicyService`) says *what* can be done; the class (`PolicyServiceImpl`) says *how*. The controller depends only on the interface, so the implementation can be swapped or mocked. An interface has no fields and no constructor; only the class does.

5.7 Dependency injection and the constructor

```
private final PolicyRepository policyRepository;

public PolicyServiceImpl(PolicyRepository policyRepository) {
    this.policyRepository = policyRepository;
}
```

Spring sees `@Service`, sees the constructor needs a repository, builds or finds one, and calls the constructor. We never write `new` for it.

Why constructor injection: the field can be `final`; tests can pass a fake repository straight to the constructor; missing dependencies fail at startup; all dependencies are visible in one place.

Why not `new`: a repository is an interface (cannot be created with `new`); real dependencies need their own dependencies; Spring shares one instance; annotations such as `@Transactional` and `@Cacheable` only work on Spring-made proxies and are *silently ignored* on a plain `new` object; tests could not substitute fakes. This is **Inversion of Control**. Use `new` for data (DTOs, entities), inject long-lived helpers (services, repositories, clients).

Rule of thumb: if a class calls another Spring object, add a `private final` field, one constructor parameter, and one `this.x = x;` line.

5.8 `@Transactional`

A transaction groups database steps so that *all succeed or none are saved*. Spring wraps the method in a proxy: start, then commit on normal return or roll back on a `RuntimeException`.

Method	Annotation	Why
`create`	`@Transactional`	It writes; nothing is half-saved on failure.
`list`, `check`	`@Transactional(readOnly = true)`	Read only; lets Spring and Hibernate skip write work.

Put it on **service** methods only. Pitfall (common interview question): calling a `@Transactional` method from another method *in the same class* bypasses the proxy and starts no transaction. It matters most later, when "set status COMPLETED *and* write the outbox row" must be one transaction.

5.9 `extends` vs `implements`; inheritance vs polymorphism

	extends	**implements**
Reads as	"is a"	"also can do"
Parent is	a class (or an interface, if the child is an interface)	an interface
How many	one class	many interfaces
Our code	`PolicyRepository extends JpaRepository`	`PolicyServiceImpl implements PolicyService`

Inheritance means "I get your methods". **Polymorphism** is what it allows: a variable of the parent type holds a child object and the child's behavior runs. The controller holds a `PolicyService`; Spring supplies a `PolicyServiceImpl`.

5.10 SQL types used

`BIGSERIAL`	Auto-increment 64-bit number (`Long`). The database picks the id; entity uses `IDENTITY`.
`TIMESTAMPTZ`	Date and time with time zone, stored in UTC (`Instant`). Avoids wrong times across servers.
`JSONB`	Binary JSON that Postgres can query and index. Fits rules whose shape varies. Entity: `Map<String,Object>` with `@JdbcTypeCode(SqlTypes.JSON)`.

5.11 Small but useful

- `Map.of()` builds a small *read-only* map that cannot contain `null`. It is a *value*, not a type: the field stays `Map<String,Object>`. We use it so `conditions` is `{}` instead of `null` (the column is NOT NULL).
- `stream().anyMatch(p -> ...)` returns true if at least one element passes the test.
- Flyway files are named `V1__name.sql`; never edit an applied file, add `V2__...` instead.
- Spring 6 no longer matches `/policies/` for `/policies`; map the class path and leave method paths empty.
- A parent-POM `spring-boot-maven-plugin` breaks a module with no main class; add it after writing `@SpringBootApplication`.

6 Mistakes and lessons

Mistake	Lesson
`.properties` lines in a `.yml` file	Use nested YAML (or rename to `.properties`); keep one config file.
`SpringBootApplication.run`	The method is on `SpringApplication`; the annotation has no `run`.
`class PolicyRepository extends JpaRepository`	A repository must be an `interface`.
`tennatId` typo	Field names drive column names and query method names; `validate` catches it.
Local variable instead of a field for the repository	Dependencies are fields set by the constructor.
Returning on the first rule in a loop	Security logic must not depend on row order; DENY first.
Default allow when no rule matched	Zero trust: default deny.
Two `return` statements and a wrong return type	One return per path; return what the service gives back.

7 Next steps

Remaining in Week 1 (about 3–4 hours, ends around 28 October)

1. `GlobalExceptionHandler` with `@RestControllerAdvice`, returning `ProblemDetail` (RFC 9457) instead of a bare 400.
2. Caffeine `@Cacheable` on `check`, since it runs on every agent action.
3. Three Mockito tests: allow, deny, no matching rule (plus "ALLOW and DENY gives deny").
4. Commit in small pieces, and write the three-line daily note.

Upcoming

- Week 2: Auth Service (Spring Authorization Server, token exchange) and Gateway. The Policy Service will accept only valid JWTs and check `act.sub` equals `agentId`.
- Week 3: Budget Service, the core of the project (Redis Lua, race test). First checkpoint on 15 November.

8 Interview answers from today

1. **What does the Policy Service do?**
   "It decides if an agent can use a tool. A DENY rule always beats an ALLOW rule, and if no rule matches, the answer is deny. That is zero trust: nothing is allowed unless someone said so."
2. **Why DTOs and not entities?**
   "DTOs hide the database design, stop clients from setting fields like id and timestamps, give each endpoint the shape it needs, and keep validation on the input."

3. **Why constructor injection?**
   "Dependencies can be final, tests can pass fakes without Spring, and a missing dependency fails at startup. Using new would also bypass Spring proxies, so @Transactional would be silently ignored."

4. **What does @Transactional do?**
   "It wraps the method so all database steps commit together or roll back together. Reads use readOnly."